

Project 0 - ASEAN Phonebook

1. Project Overview

A university international-relations office keeps a working phonebook of students and partner contacts from different Southeast Asian countries. Contacts may be added, searched, updated, deleted, and displayed while the program is running.

The important part of this project is not the contact information by itself. The important part is **how the contacts are stored and maintained**.

You will build the phonebook using a **manually implemented singly linked list**.

The phonebook must:

- store every contact inside a `Node`;
- keep the contacts in sorted order;
- reject duplicate student IDs;
- reject duplicate complete phone numbers;
- allow contacts to be searched, updated, deleted, and listed;
- preserve the linked-list structure after every input line is processed.

Python's normal `list` must **not** be used as the main storage of phonebook contacts. The contacts must also **not** be arranged using `sort()` or `sorted()`.

This is a console-based Python project. There is no database, GUI, web application, or permanent file storage. The phonebook exists only while the program is running.

2. What You Will Build

The scaffold is divided into four Python files:

```
constants.py
contact.py
phonebook.py
main.py
```

Each file has one clear purpose:

- `constants.py` contains the provided country-code dictionary and fixed update-field names;
- `contact.py` contains `Contact` and the contact-validation helpers;
- `phonebook.py` contains `Node` and `Phonebook`;
- `main.py` reads the complete input, identifies each input-line type, calls the correct phonebook operation, and prints the required output.

The three main objects are still:

- `Contact` — represents one person;
- `Node` — stores one `Contact` and a link to the next node;
- `Phonebook` — manages the complete singly linked list.

The program will read input lines such as:

```
ADD|S-001|Abad|Lio|Analyst|62|021|000777
FIND|S-001
UPDATE|S-001|OCCUPATION|Systems Analyst
DELETE|S-001
LIST
```

Each input line asks the phonebook to perform one operation.

The linked list is the authoritative collection. Searching, adding, updating, deleting, filtering, and listing must work from that linked structure.

3. Program Structure - Who Does What?

The scaffold already separates the program into files. Keep the required classes and functions in their provided files so each part of the program has one clear responsibility.

3.1 `contact.py` — `Contact`

A `Contact` stores the information for one person.

It contains these fields:

```
student_id
surname
given_name
occupation
country_code
area_code
local_number
```

`Contact` is also responsible for its own readable string output through `__str__()`.

The required `__str__()` format is:

```
<student_id> - <surname>, <given_name> - <occupation> - <country_name> - <country_code>-<area_code>-<local_number>
```

Example:

```
S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777
```

`Contact` should **not**:

- store `next`;
- search the phonebook;
- add or delete nodes;
- read program input lines.

3.2 `phonebook.py` — `Node`

A `Node` represents one position in the singly linked list.

It stores exactly:

```
contact
next
```

where:

- `contact` refers to one `Contact`;
- `next` refers to the next `Node`, or `None` if there is no next node.

A `Node` should **not**:

- read input;
- print results;
- search the whole phonebook;
- decide where another node should be inserted.

3.3 `phonebook.py` — `Phonebook`

`Phonebook` owns the linked list.

At minimum, it stores:

```
head
size
```

It is responsible for:

- adding contacts;
- finding contacts;
- finding contacts by surname;
- updating contacts;
- deleting contacts;
- listing contacts;
- filtering contacts by country;
- keeping the contacts sorted;
- keeping `size` correct;
- linking and unlinking `Node` objects.

The linked-list operations belong here.

3.4 `main.py` — Input Processor

The input-processing part of the scaffold is responsible for:

1. reading each input line;
2. separating the line into its fields;
3. identifying the requested input type;
4. calling the correct `Phonebook` method;
5. printing the required result.

The input processor must **not** directly change:

```
node.next
phonebook.head
```

Those changes belong inside `Phonebook`.

3.5 `constants.py` — Country Lookup and Fixed Values

`constants.py` is already complete. It contains fixed values used by the project, including the full country-code dictionary:

```
from typing import Final

COUNTRY_CODES: Final[dict[str, str]] = {
    "60": "Malaysia",
    "62": "Indonesia",
    "63": "Philippines",
    # the remaining required codes are already provided
}
```

It also contains `UPDATE_FIELDS`, the allowed field names used by `UPDATE`.

`main.py` imports `COUNTRY_CODES`, so the dictionary is directly available while input lines are being processed. `contact.py` imports the same dictionary when `Contact.__str__()` needs the country name.

Do not duplicate or replace these fixed values in another file. The country dictionary is only for translating and validating country codes. It must not be used as another place to store contacts.

4. Linked List Rules

After every valid or invalid input line is processed, the phonebook must still follow these rules:

1. `head` points to the first `Node`, or `None` when the phonebook is empty.
2. Every `Node` stores exactly one `Contact`.
3. Every `Node` has one `next` reference.
4. The last node has `next = None`.
5. `size` equals the actual number of contacts in the linked list.
6. No two contacts may have the same `student_id`.

7. No two contacts may have the same complete phone number.
8. Contacts remain sorted by surname, then given name, then `student_id`.
9. Sorting must be maintained through linked-list insertion or reinsertion. Do not use `sort()` or `sorted()`.

4.1 How Contacts Are Sorted

Contacts are arranged using:

```
(contact.surname.lower(), contact.given_name.lower(), contact.student_id)
```

The original spelling and capitalization must still be stored and displayed. `.lower()` is used only when comparing names.

Example 1 - Different surnames

Abad, Lio

Garcia, Mina

Abad comes before Garcia.

Example 2 - Same surname

Lim, Ana

Lim, Bea

Both surnames are the same, so the given name is checked next. **Ana** comes before **Bea**.

Example 3 - Same surname and given name

S-001 | Lim, Ana

S-009 | Lim, Ana

The surname and given name are both the same. The `student_id` becomes the final comparison, so S-001 comes first.

This last comparison prevents two contacts with the same name from having an unclear order.

4.2 Duplicate Contact Rules

A student ID must be unique.

A complete phone number is also unique. The complete phone number is the combination of:

`country_code-area_code-local_number`

Example 1 - Duplicate student ID Existing contact:

S-001 - Abad, Lio

New contact:

S-001 - Garcia, Mina

Result:

ERROR DUPLICATE_ID S-001

The names are different, but the student ID already exists.

Example 2 - Duplicate complete phone number Existing phone:

63-02-001234

New contact also uses:

63-02-001234

Result:

ERROR DUPLICATE_PHONE 63-02-001234

Example 3 - Same local number but different complete phone number Existing phone:

63-02-001234

New phone:

60-03-001234

This is allowed because the complete phone numbers are different.

4.3 Country Codes

Use only the following country table.

Calling code	Country name
60	Malaysia
62	Indonesia
63	Philippines
65	Singapore
66	Thailand
84	Vietnam
95	Myanmar
670	Timor-Leste
673	Brunei Darussalam
855	Cambodia
856	Lao PDR

`country_code`, `area_code`, and `local_number` are stored as strings.

This is important because leading zeros must remain unchanged.

Example 1 Input:

63

Country:

Philippines

Example 2 Phone values:

60

03

000111

Displayed phone:

60-03-000111

The zeros in 03 and 000111 must not disappear.

Example 3 - Unknown country code Input:

999

Result:

ERROR INVALID_COUNTRY 999

5. Python and Linked List Requirements

This project uses **Python only**.

The following rules are required:

- Implement **Node** manually.
- Implement the singly linked list manually.
- The main phonebook contacts must be stored through **Node** objects beginning from **head**.
- Do not use a Python **list** as the main contact storage.
- Do not use **list.sort()**.
- Do not use **sorted()**.
- Do not copy all contacts into a Python list just to sort them and rebuild the linked list.
- A Python list created temporarily by operations such as **split()** is allowed for parsing input.
- A dictionary is allowed for the fixed country table.
- Helper functions and methods are allowed when they make the code easier to understand.
- Do not place every operation inside one very large function.

The purpose of the project is to practice maintaining the linked list itself. A solution that replaces the linked list with a Python list does not satisfy the project requirement.

6. Input Format

6.1 Version Line

The first line must be:

```
ASEAN-PHONEBOOK 1.0
```

If this line is different, print:

```
ERROR VERSION
```

and stop.

6.2 Number of Input Lines

The second line tells the program how many phonebook input lines follow.

Example:

```
ASEAN-PHONEBOOK 1.0
5
ADD|S-001|Abad|Lio|Analyst|62|021|000777
FIND|S-001
UPDATE|S-001|OCCUPATION|Systems Analyst
LIST
DELETE|S-001
```

The value 5 means exactly five input lines must be processed after it.

The program processes only the declared number of input lines. If fewer lines are provided than the declared number, print **ERROR COMMAND_COUNT** and stop. Any text after the declared number of input lines is outside the phonebook input and is ignored.

The number must be a whole number from 0 to 200.

Valid examples:

```
0
5
120
```

Invalid examples:

-1
+5
005
201

If the value is missing or invalid, print:

ERROR COMMAND_COUNT

and stop.

6.3 Contact Fields

An ADD input line contains:

ADD|student_id|surname|given_name|occupation|country_code|area_code|local_number

The fields follow these rules:

Field	Rule
student_id	1-20 characters; uppercase A-Z, numbers 0-9, and -; first character must be a letter or number
surname	1-40 characters; letters A-Z or a-z, spaces, apostrophes, and hyphens; no leading or trailing space
given_name	Same rule as surname
occupation	1-60 characters; letters, numbers, spaces, periods, apostrophes, hyphens, and /; no leading or trailing space
country_code	Must exist in the country table
area_code	1-6 digits; stored as text
local_number	3-12 digits; stored as text

The character | is used to separate fields and therefore cannot appear inside a field value.

For an invalid field value, print:

ERROR INVALID_VALUE <FIELD>

Example:

ERROR INVALID_VALUE STUDENT_ID

For an unknown country code, print:

ERROR INVALID_COUNTRY <value>

6.4 If One Input Line Has More Than One Problem

Sometimes one input line can contain more than one mistake.

The program must report only one error, so checks happen in a fixed order.

Example 1 - Wrong number of fields Input:

ADD|S-001|Abad|Lio|Analyst|999

The line does not contain all required ADD fields.

Even though 999 is also not a valid country code, the field count is checked first.

Output:

ERROR MALFORMED ADD

Example 2 - Duplicate ID and duplicate phone at the same time Suppose this contact already exists:

A-1 - Lim, Ana - Coordinator - Malaysia - 60-03-000111

Then the program receives another ADD using both the same ID and the same phone.

The duplicate ID is checked first.

Output:

ERROR DUPLICATE_ID A-1

Example 3 - UPDATE target is missing Input:

UPDATE|ZZ-9|BANANA|Something

BANANA is not a valid update field, but ZZ-9 does not exist.

The program must first find the requested contact.

Output:

ERROR NOT_FOUND ZZ-9

For consistency, use this general checking order:

1. check whether the input line has the correct number of fields;
2. find the requested contact first when an operation needs an existing contact;
3. validate field values from left to right;
4. for ADD, check duplicate ID before duplicate phone;
5. for UPDATE, validate the complete proposed change before changing the linked list;
6. if an operation fails, the phonebook must remain unchanged.

An empty input line prints:

ERROR MALFORMED

An unrecognized input type prints:

ERROR UNKNOWN_COMMAND <word>

7. Input Line Types and Required Output

7.1 ADD

Input:

ADD|student_id|surname|given_name|occupation|country_code|area_code|local_number

A valid contact must be inserted directly into its correct linked-list position.

Success:

OK ADD <student_id>

Possible duplicate errors:

ERROR DUPLICATE_ID <student_id>

ERROR DUPLICATE_PHONE <country_code>-<area_code>-<local_number>

Example:

ADD|S-002|Garcia|Mina|Designer|63|02|001234

Output:

OK ADD S-002

7.2 FIND

Input:

FIND|student_id

If the contact exists:

FOUND | <Contact.__str__() output>

Example:

FIND|S-001

Output:

FOUND | S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777

If the contact does not exist:

ERROR NOT_FOUND <student_id>

7.3 FIND_SURNAME

Input:

FIND_SURNAME|surname

Surname matching ignores capitalization.

For example:

FIND_SURNAME|GARCIA

matches stored surnames such as:

Garcia

garcia

GARCIA

Print the number of matches first:

MATCHES <count>

Then print each matching contact in the same order in which the contacts appear in the linked list:

CONTACT | <Contact.__str__() output>

Example:

MATCHES 2

CONTACT | S-002 - Garcia, Mina - Designer - Philippines - 63-02-001234

CONTACT | S-010 - Garcia, Noel - Analyst - Malaysia - 60-03-000999

When there are no matches:

MATCHES 0

7.4 UPDATE

Input:

UPDATE|target_student_id|FIELD|new_value

FIELD must be one of:

ID

SURNAME

GIVEN_NAME

OCCUPATION

COUNTRY_CODE

AREA_CODE
LOCAL_NUMBER

A successful update prints:

OK UPDATE <FIELD> | <old_value> -> <new_value>

Examples:

OK UPDATE SURNAME | Garcia -> Abao
OK UPDATE OCCUPATION | Designer -> Systems Analyst
OK UPDATE ID | S-002 -> S-010

Updating a field to its current value is still a valid update.

If ID, SURNAME, or GIVEN_NAME changes, the contact may belong in a different sorted position. In that case, remove the contact's node from its old position and reinsert it into the correct linked-list position.

Do not use `sort()` or `sorted()` after an update.

Possible errors include:

ERROR NOT_FOUND <target_student_id>
ERROR INVALID_FIELD <FIELD>
ERROR INVALID_VALUE <FIELD>
ERROR INVALID_COUNTRY <value>
ERROR DUPLICATE_ID <new_id>
ERROR DUPLICATE_PHONE <candidate_phone>

A failed update must leave the original contact unchanged.

7.5 DELETE

Input:

DELETE|student_id

Success:

OK DELETE <student_id>

Missing contact:

ERROR NOT_FOUND <student_id>

Deleting a contact must correctly handle:

- deleting the only node;
- deleting `head`;
- deleting a middle node;
- deleting the last node.

`size` must also be updated.

7.6 LIST

Input:

LIST

First print:

LIST <count>

Then print every contact in linked-list order:

CONTACT | <Contact.__str__() output>

Example:

LIST 2

CONTACT | S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777

CONTACT | S-002 - Garcia, Mina - Designer - Philippines - 63-02-001234

For an empty phonebook:

LIST 0

No CONTACT line follows LIST 0.

7.7 COUNTRY

Input:

COUNTRY|code[,code...]

Examples:

COUNTRY|63

COUNTRY|63,60

COUNTRY|60,62,63

When all requested codes are valid, print:

COUNTRY_MATCHES <count>

Then print each matching contact in the same global linked-list order:

CONTACT | <Contact.__str__() output>

Example:

COUNTRY_MATCHES 2

CONTACT | S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777

CONTACT | S-002 - Garcia, Mina - Designer - Philippines - 63-02-001234

Repeated country codes in the input are treated only once.

Example:

COUNTRY|63,63,60

is treated the same as:

COUNTRY|63,60

If the country list is empty or malformed:

ERROR INVALID_VALUE COUNTRY_CODES

If an unknown country code appears, print the first unknown code:

ERROR INVALID_COUNTRY <code>

8. Sample 1 - Adding, Updating, and Sorted Order

Input

ASEAN-PHONEBOOK 1.0

8

ADD|S-002|Garcia|Mina|Designer|63|02|001234

ADD|S-001|Abad|Lio|Analyst|62|021|000777

LIST

FIND_SURNAME|GARCIA

UPDATE|S-002|SURNAME|Abao

LIST

```
DELETE|S-001
LIST
```

Output

```
OK ADD S-002
OK ADD S-001
LIST 2
CONTACT | S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777
CONTACT | S-002 - Garcia, Mina - Designer - Philippines - 63-02-001234
MATCHES 1
CONTACT | S-002 - Garcia, Mina - Designer - Philippines - 63-02-001234
OK UPDATE SURNAME | Garcia -> Abao
LIST 2
CONTACT | S-001 - Abad, Lio - Analyst - Indonesia - 62-021-000777
CONTACT | S-002 - Abao, Mina - Designer - Philippines - 63-02-001234
OK DELETE S-001
LIST 1
CONTACT | S-002 - Abao, Mina - Designer - Philippines - 63-02-001234
```

Notice that changing Garcia to Abao changes the contact's sorted position. The linked list must be repaired by unlinking and reinserting that contact in the correct place.

9. Sample 2 - Errors and Unchanged Data

Input

```
ASEAN-PHONEBOOK 1.0
9
LIST
ADD|A-1|Lim|Ana|Coordinator|60|03|000111
ADD|B-2|Santos|Bea|Technician|63|082|009999
ADD|A-1|Other|Name|Writer|65|01|123456
ADD|C-3|Thirid|Name|Writer|63|082|009999
UPDATE|A-1|ID|B-2
UPDATE|A-1|COUNTRY_CODE|999
DELETE|ZZ-9
LIST
```

Output

```
LIST 0
OK ADD A-1
OK ADD B-2
ERROR DUPLICATE_ID A-1
ERROR DUPLICATE_PHONE 63-082-009999
ERROR DUPLICATE_ID B-2
ERROR INVALID_COUNTRY 999
ERROR NOT_FOUND ZZ-9
LIST 2
CONTACT | A-1 - Lim, Ana - Coordinator - Malaysia - 60-03-000111
CONTACT | B-2 - Santos, Bea - Technician - Philippines - 63-082-009999
```

The failed ADD, UPDATE, and DELETE input lines do not change the valid contacts already stored in the phonebook.

10. Milestone Plan

The project can be completed in the following order.

Milestone	What should work before moving on
1. Contact Model	<code>constants.py</code> is left unchanged; <code>Contact</code> and its validation helpers in <code>contact.py</code> work correctly, including <code>__str__()</code> .
2. Linked List Foundation	<code>Node</code> and <code>Phonebook</code> in <code>phonebook.py</code> , including <code>head</code> , <code>size</code> , and basic traversal, work correctly.
3. Adding Contacts	<code>ADD</code> rejects duplicates and performs manual sorted insertion for an empty list, head, middle, and tail.
4. Reading Contacts	<code>FIND</code> , <code>FIND_SURNAME</code> , <code>LIST</code> , and <code>COUNTRY</code> traverse the linked list and produce the required output.
5. Changing Contacts	<code>UPDATE</code> and <code>DELETE</code> work without corrupting links. Key-changing updates reinsert contacts when needed.
6. Final Integration	<code>main.py</code> correctly connects the input contract to the completed <code>Contact</code> and <code>Phonebook</code> classes, and all supplied tests pass.

Do not skip the linked-list foundation and attempt to solve the project by storing contacts in a Python list first. The later operations are intended to build on the linked structure created in the earlier milestones.

11. Scaffold and Submission Requirements

The student package contains:

```
constants.py
contact.py
phonebook.py
main.py
public_tests/
    README.md
    run_tests.py
    test_structure.py
cases/
```

Required Python Files

All four Python files are part of the scaffold. Their required classes, constructors, functions, type hints, constants, docstrings, imports, and program entry point are already provided. Assessed function bodies use `pass` where students are expected to implement the behavior.

Keep each required component in its provided file:

```
constants.py  COUNTRY_CODES and UPDATE_FIELDS
contact.py    Contact and contact-validation helpers
phonebook.py  Node and Phonebook
main.py       input parsing, run_program(), and main()
```

`constants.py` is already complete. Students should not change its country table or update-field names.

Do not rename the required files, classes, functions, methods, or constants. Do not combine the files back into one source file.

Public Tests

The `public_tests` folder is provided for preliminary checking. Do not edit the expected output files to make an incorrect program appear correct.

Run the complete public test package using:

```
python public_tests/run_tests.py
```

The public package checks the required file/class separation, classes/functions directly, and the published input/output behavior. Passing every public test is a strong indication that the documented project behavior works correctly. Additional in-scope edge cases remain part of instructor validation.

The public tests are not part of the required submission.

What to Submit

Submit one ZIP file named:

```
<student_id>_P0.zip
```

The ZIP must contain only:

```
constants.py  
contact.py  
phonebook.py  
main.py
```

No separate test plan, complexity report, design report, or additional documentation is required. Do not include `public_tests`, virtual environments, IDE cache folders, compiled files, or unrelated files in the submission ZIP.

12. Individual Work

Project 0 is strictly individual.

Each student must submit their own completed four-file scaffold and should be able to explain the linked-list operations used in their solution.

The project is assessed through the required program behavior and the provided test cases. The main requirement is that the phonebook genuinely uses the manually implemented singly linked list described in this project definition.